

Herramientas De Versionamiento (GIT) Instalada Y Configurada

GA7-220501096-AA1-EV05

Autor:

- Yeison Adolfo Díaz Tapasco

Institución:

Servicio Nacional de Aprendizaje (SENA)

Programa: Análisis y Desarrollo de Software

Profesor:

Oscar Fernando Carvajal

Abril 2026

TABLA DE CONTENIDO

1. Introducción
2. Objetivos
3. Herramientas y requisitos
4. Creación del entorno de trabajo
5. Instalación y verificación de Git
6. Desarrollo del ejercicio
7. Conclusiones

1. INTRODUCCIÓN

En el desarrollo de software moderno, la gestión del código fuente es fundamental para garantizar la trazabilidad, el control de cambios y la colaboración entre desarrolladores. En este contexto, Git se posiciona como una de las herramientas más utilizadas para el control de versiones, permitiendo registrar y gestionar modificaciones tanto en entornos locales como remotos.

La presente evidencia tiene como propósito implementar y validar el uso de Git mediante la ejecución de comandos fundamentales, así como su integración con un repositorio remoto en GitHub. A través de un ejercicio práctico, se evidencia el flujo completo de trabajo, desde la creación del repositorio hasta la sincronización de cambios en la nube.

Este proceso permite comprender la importancia del versionamiento en el ciclo de vida del desarrollo de software y su aplicación en escenarios reales.

2. OBJETIVOS

2.1 Objetivo general

Implementar el uso de Git como herramienta de control de versiones, mediante la ejecución de comandos en un entorno local y su integración con un repositorio remoto.

2.2 Objetivos específicos

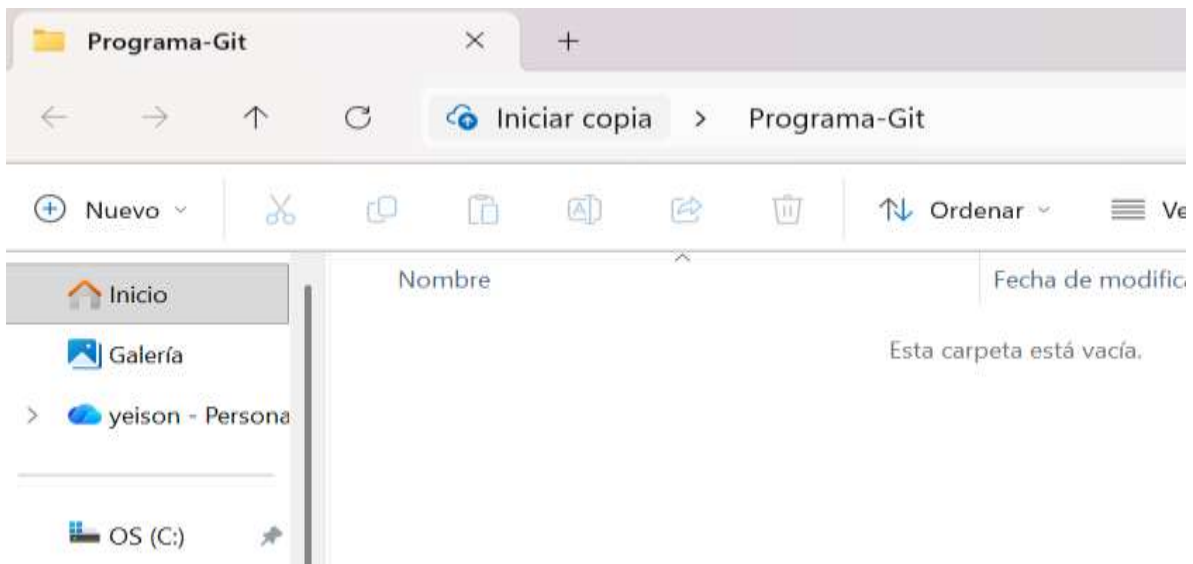
- Verificar la correcta instalación de Git en el sistema.
- Crear y gestionar un repositorio local.
- Configurar un repositorio remoto en GitHub.
- Establecer la conexión entre repositorio local y remoto.
- Ejecutar comandos fundamentales de Git (init, add, commit, push).
- Validar la sincronización de archivos entre ambos entornos.

3. HERRAMIENTAS Y REQUISITOS

- Windows
- CMD
- Git
- Cuenta en GitHub
- Internet

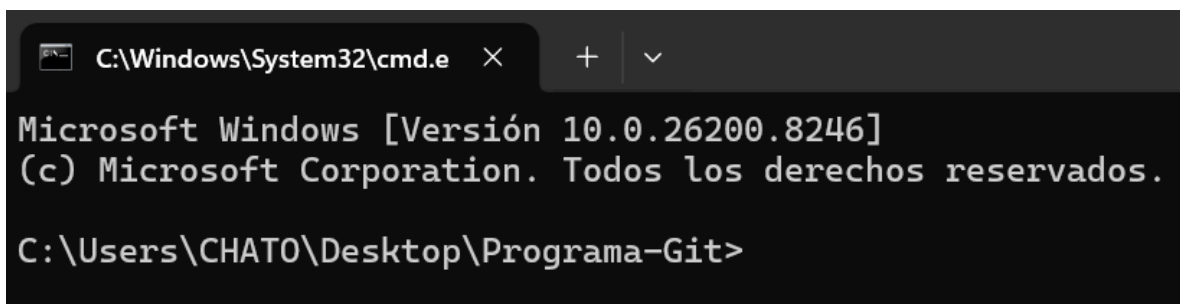
4. CREACIÓN DEL ENTORNO DE TRABAJO

Figura 1. Carpeta del proyecto



Se crea una carpeta en el escritorio denominada Programa-Git, la cual funcionará como repositorio local para la gestión y control de versiones del proyecto.

Figura 2. CMD en carpeta

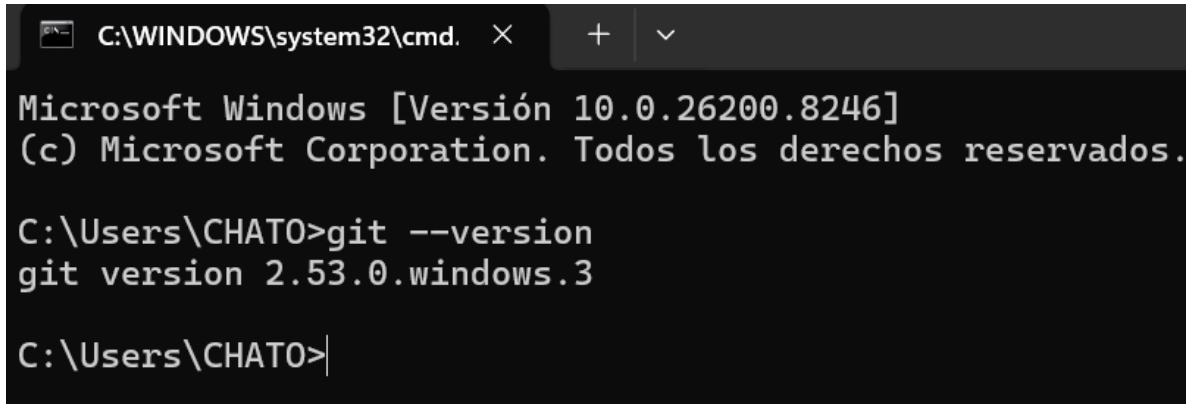


Se abre el símbolo del sistema (CMD) dentro de la carpeta del proyecto, lo que permite ejecutar los comandos de Git directamente en la ruta donde se almacenarán los archivos.

5. INSTALACIÓN Y VERIFICACIÓN DE GIT

Figura 3. Verificación de instalación de Git

git --version



```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Versión 10.0.26200.8246]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\CHATO>git --version
git version 2.53.0.windows.3

C:\Users\CHATO>
```

Se verifica la correcta instalación de Git en el sistema mediante el comando `git --version`, el cual permite confirmar que la herramienta está disponible para su uso.

6. DESARROLLO DEL EJERCICIO

6.1 Inicialización del repositorio

git init

Figura 4. Inicialización del repositorio



```
C:\Users\CHATO\Desktop\Programa-Git>git init
Initialized empty Git repository in C:/Users/CHATO/Desktop/Programa-Git/.git/
C:\Users\CHATO\Desktop\Programa-Git>
```

Se inicializa el repositorio local mediante el comando git init, lo que permite comenzar el control de versiones sobre los archivos del proyecto.

6.2 Creación de archivo y primer commit

echo Proyecto Git > README.md

git add .

git commit -m "Primer commit"

Figura 5. Archivo creado

```
C:\Users\CHAT0\Desktop\Programa-Git>echo Proyecto Git > README.md

C:\Users\CHAT0\Desktop\Programa-Git>dir
El volumen de la unidad C es OS
El número de serie del volumen es: 327D-26BD

Directorio de C:\Users\CHAT0\Desktop\Programa-Git

22/04/2026  11:05 a. m.    <DIR>          .
22/04/2026  10:55 a. m.    <DIR>          ..
22/04/2026  11:05 a. m.                  15 README.md
                1 archivos                15 bytes
                2 dirs 677.782.286.336 bytes libres

C:\Users\CHAT0\Desktop\Programa-Git>
```

Se crea el archivo README.md mediante comandos en la consola, el cual sirve como archivo inicial del proyecto para almacenar información básica.

Figura 6. Primer commit

```
C:\Users\CHAT0\Desktop\Programa-Git>git commit -m "Primer commit"
[master (root-commit) 867f58e] Primer commit
1 file changed, 1 insertion(+)
create mode 100644 README.md

C:\Users\CHAT0\Desktop\Programa-Git>
```

Se agregan los archivos al área de preparación con git add . y se realiza el primer commit con git commit, registrando así el estado inicial del proyecto en el historial de versiones.

6.3 Creación del repositorio remoto en GitHub

Figura 7. Repositorio remoto



Se crea un repositorio remoto en GitHub, el cual permitirá almacenar y sincronizar los archivos del proyecto en la nube.

6.4 Conexión entre repositorios

```
git remote add github https://github.com/yd051197-web/Programa-Git.git
```

```
git remote -v
```

Figura 8. Conexión remota



```
C:\Users\CHAT0\Desktop\Programa-Git>git remote -v
github  https://github.com/yd051197-web/Programa-Git.git (fetch)
github  https://github.com/yd051197-web/Programa-Git.git (push)

C:\Users\CHAT0\Desktop\Programa-Git>|
```

Se establece la conexión entre el repositorio local y el repositorio remoto mediante el comando `git remote`, permitiendo la sincronización entre ambos entornos.

6.5 Envío de cambios (push)

```
git branch -M main
```

```
git push -u github main
```

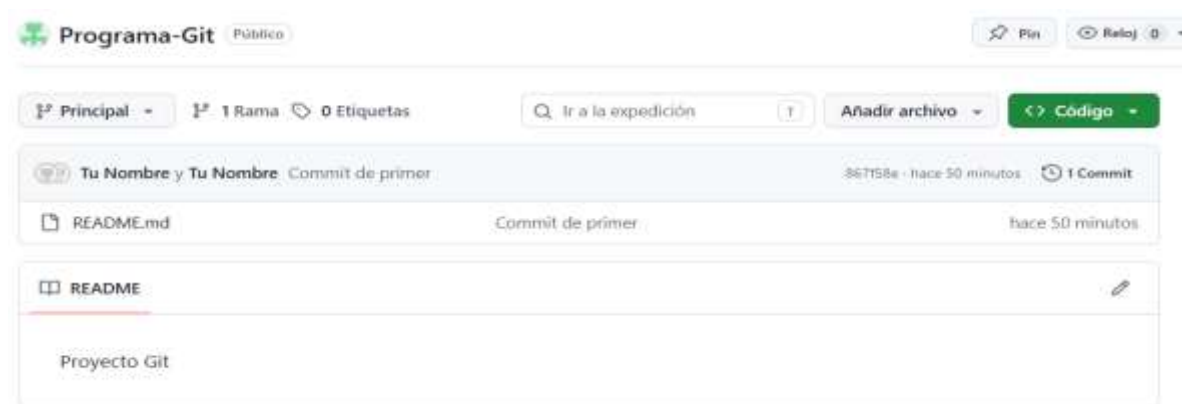
Figura 9. Push inicial

```
C:\Users\CHAT0\Desktop\Programa-Git>git branch -M main

C:\Users\CHAT0\Desktop\Programa-Git>git push -u github main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 217 bytes | 108.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/yd051197-web/Programa-Git.git
 * [new branch]      main -> main
branch 'main' set up to track 'github/main'.
```

Se envían los archivos del repositorio local al repositorio remoto mediante el comando git push, permitiendo almacenar los cambios en la plataforma remota.

Figura 10. Archivo en GitHub



Se verifica en la plataforma GitHub que los archivos han sido cargados correctamente, confirmando la sincronización.

6.6 Creación de archivo kotlin.txt

notepad kotlin.txt

Nombre: Yeison Díaz

Correo: ydo51197@gmail.com

Figura 11. Archivo kotlin.txt

```
C:\Users\CHAT0\Desktop\Programa-Git>dir
El volumen de la unidad C es OS
El número de serie del volumen es: 327D-26BD

Directorio de C:\Users\CHAT0\Desktop\Programa-Git

22/04/2026  12:01 p. m.    <DIR>          .
22/04/2026  10:55 a. m.    <DIR>          ..
22/04/2026  12:05 p. m.                48 kotlin.txt
22/04/2026  11:05 a. m.                15 README.md
                2 archivos                63 bytes
                2 dirs 677.817.147.392 bytes libres

C:\Users\CHAT0\Desktop\Programa-Git>|
```

Se crea el archivo kotlin.txt mediante el bloc de notas, en el cual se ingresan manualmente los datos del usuario (nombre y correo electrónico), y se guarda dentro del repositorio local.

6.7 Commit y sincronización final

git add .

git commit -m "Añadido datos"

git push

echo prueba video > video.txt

git add .

git commit -m "Prueba para video"

git push

Figura 12. Commit final

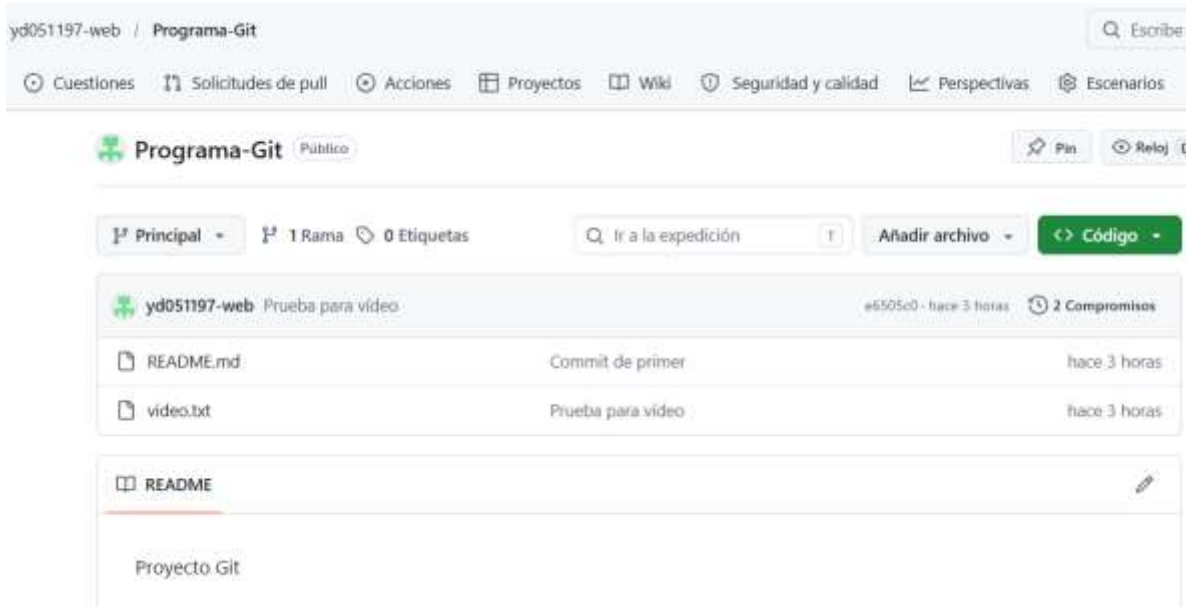
```
C:\Users\CHAT0\Desktop\Programa-Git>git add .

C:\Users\CHAT0\Desktop\Programa-Git>git commit -m "Añadido datos"
[main 82684ec] Añadido datos
 1 file changed, 2 insertions(+)
 create mode 100644 kotlin.txt

C:\Users\CHAT0\Desktop\Programa-Git>git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 321 bytes | 160.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/yd051197-web/Programa-Git.git
 867f58e..82684ec  main -> main

C:\Users\CHAT0\Desktop\Programa-Git>
```

Se registran los cambios realizados en el archivo kotlin.txt y se sincronizan con el repositorio remoto, garantizando la actualización del proyecto en la plataforma GitHub.

Figura 13. Repositorio actualizado

Se realiza un nuevo commit con los cambios realizados y se envían al repositorio remoto, evidenciando la correcta sincronización entre el entorno local y la plataforma remota.

LINK DEL VIDEO: <https://www.loom.com/share/e556374f86c84e588236dd213a67743a>

7. CONCLUSIONES

Se logró implementar Git como herramienta de control de versiones, permitiendo gestionar archivos en un entorno local y sincronizarlos con un repositorio remoto. La ejecución de comandos básicos evidenció el flujo completo de trabajo en Git, desde la inicialización del repositorio hasta la actualización de cambios en la plataforma remota. El uso de herramientas como GitHub facilita la gestión del código fuente, mejora la organización del proyecto y permite mantener un historial detallado de cambios, siendo una práctica esencial en el desarrollo de software.